

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

AIM: To Perform gray level operations images.

Theory:

Gray level operations involve manipulating the pixel values of an image to enhance or modify its appearance. These operations are commonly used in image processing tasks such as contrast adjustment, brightness correction, and image thresholding.

Flowchart:

The flowchart for the gray level operations program will consist of the following steps:
Load the input image.
Convert the image to grayscale.
Perform the desired gray level operation.
Display the processed image.
Save the processed image (optional).

CODE:

```
pip install opencv-python
```

```
#import libraries
```

```
#cv2 lib reads in this format : blue-green-red plane (bgr)
```

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
#read the input image
```

```
image = cv2.imread("images/image_1.jpg")
```

```
if image is None:
```

```
    print("Error:Image not found")
```

```
    exit()
```

```
#this will give the original image --> instead of reading in bgr plane it will read in rgb
```

```
image1 = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
```

```
#display the input image
```

```
plt.figure(figsize=(4,4))
```

```
plt.imshow(image1)
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

```
plt.title("Input image")
plt.axis("off")
plt.show()
```

Output:



GRAY LEVEL IMAGE

Code:

```
gray_image = cv2.imread("images/image_1.jpg",cv2.IMREAD_GRAYSCALE)
```

```
plt.figure(figsize=(4,4))
plt.imshow(gray_image,cmap="gray") #default-verdis : yellowish shade
plt.title("Gray image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

Gray image



Conclusion:

The input image is converted into grayscale image which represents the intensity levels between black and white.

IMAGE NEGATION

Code:

```
negat_image = 255 - gray_image
```

```
plt.figure(figsize=(4,4))
plt.imshow(negat_image,cmap="gray") #default-verdis : yellowish shade
plt.title("Negat image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

Negat image



Conclusion:

The bright regions of the gray scale image appear dark and dark regions appear bright.
The intensity values of the gray image are inverted

LOGARITHMIC TRANSFORMATION

Code:

```
#apply log transformation
c = 40
log_image = c * np.log(1+gray_image.astype(np.float32))
log_image = np.array(log_image,dtype=np.uint8)

plt.figure(figsize=(4,4))
plt.imshow(log_image,cmap="gray") #default-verdis : yellowish shade
plt.title("Log image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

Log image



Conclusion:

In this case the details of darker areas are much clearly visible

GAMMA TRANSFORMATION

Code:

#power law

#gamma>1 → dark

#gamma<1 → brighter

#gamma =1 → same

gamma = 0.5

gamma_image = np.array(255*(gray_image/255) ** gamma,dtype=np.uint8)

plt.figure(figsize=(4,4))

plt.imshow(gamma_image,cmap="gray") #default-verdis : yellowish shade

plt.title("Gamma image")

plt.axis("off")

plt.show()

Output:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

Gamma image



Conclusion:

In this particular case $\gamma = 0.5$ hence the image appears much brighter compared to the gray-scale image ($\gamma < 1$)
We can observe the effect of power-law transformation on intensity of image.

GLOBAL THRESHOLDING

Code:

```
threshold_value = 80 #remove further noise / unwanted information
thresh_value, thresh_image =
cv2.threshold(gray_image,threshold_value,255,cv2.THRESH_BINARY)
#image and value will be provided if not error
plt.figure(figsize=(4,4))
plt.imshow(thresh_image,cmap="gray") #default-verdis : yellowish shade
plt.title("Threshold image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121



Conclusion:

In this case the pixels of the gray scale image are separated into black and white regions according to their intensity.

So finally gray scale image is converted into binary image.

ADAPTIVE THRESHOLDING - GAUSSIAN

Code:

```
#ADAPTIVE THRESHOLDING (GUASSIAN)
```

```
thresh_image =
cv2.adaptiveThreshold(gray_image,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,11,2)
#image and value will be provided if not error
plt.figure(figsize=(4,4))
plt.imshow(thresh_image,cmap="gray") #default-verdis : yellowish shade
plt.title("Adaptive (GUASSIAN) Threshold image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group NAAC A+	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology		
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.		
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121	

Adaptive (GUASSIAN) Threshold image



Conclusion:

In this case different regions are processed according to their local intensity and the threshold value is locally determined

ADAPTIVE THRESHOLDING – MEAN

Code:

```
#ADAPTIVE THRESHOLDING (MEAN)
```

```
thresh_image =
cv2.adaptiveThreshold(gray_image,255,cv2.ADAPTIVE_THRESH_MEAN_C,cv2.THRESH
H_BINARY,11,2)
#image and value will be provided if not error
plt.figure(figsize=(4,4))
plt.imshow(thresh_image,cmap="gray") #default-verdis : yellowish shade
plt.title("Adaptive (MEAN) Threshold image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

Adaptive (MEAN) Threshold image



Conclusion:

In this case different regions are processed according to their local intensity and the threshold value is locally determined.

OTSU THRESHOLDING

Code:

```
#OTSU THRESHOLDING
```

```
_,thresh_image = cv2.threshold(gray_image,0,255,cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
#image and value will be provided if not error
plt.figure(figsize=(4,4))
plt.imshow(thresh_image,cmap="gray") #default-verdis : yellowish shade
plt.title("OTSU Threshold image")
plt.axis("off")
plt.show()
```

Output:

 Marwadi University Marwadi Chandarana Group	NAAC A+	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)		AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121	



Conclusion:

In this method the threshold is automatically determined and the gray-scale image is converted into binary image

IMAGE PROPERTIES

Code:

```
print("Shape:",image.shape)
print("Height:",image.shape[0])
print("Width:",image.shape[1])
print("Channels:",image.shape[2])
print("Data type:",image.dtype)
```

Output:

```
Shape: (371, 508, 3)
Height: 371
Width: 508
Channels: 3
Data type: uint8
```

RGB CHANNEL SPLITTING

Code:

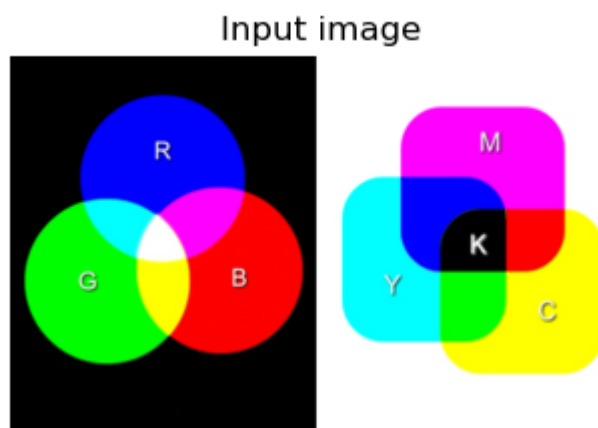
```
image_1 = cv2.imread("images/rgb.jpg")
```

```
#display the input image
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

```
plt.figure(figsize=(4,4))
plt.imshow(image_1)
plt.title("Input image")
plt.axis("off")
plt.show()
```

Output:



Code:

```
r,g,b = cv2.split(image_1)

plt.figure(figsize=(16,4))

plt.subplot(1,4,1)
plt.imshow(image_1) #default-verdis : yellowish shade
plt.title("ORIGINAL")
plt.axis("off")

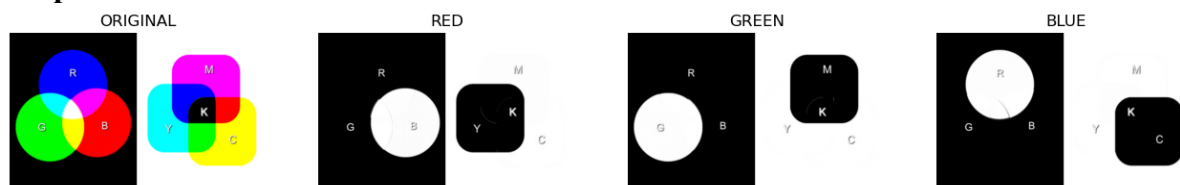
plt.subplot(1,4,2)
plt.imshow(r,cmap="gray") #default-verdis : yellowish shade
plt.title("RED")
plt.axis("off")

plt.subplot(1,4,3)
plt.imshow(g,cmap="gray") #default-verdis : yellowish shade
plt.title("GREEN")
plt.axis("off")
```

 Marwadi University Marwadi Chandarana Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Perform gray level operations images.	
Experiment No: 7	Date:13-08-2026	Enrollment No:92400133121

```
plt.subplot(1,4,4)
plt.imshow(b,cmap="gray") #default-verdis : yellowish shade
plt.title("BLUE")
plt.axis("off")
plt.show()
```

Output:



Conclusion:

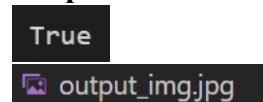
The RGB image is successfully split.
The intensity of each color component is observed individually.

OUTPUT_IMAGE_EXTRACTION

Code:

```
cv2.imwrite(r'C:\Users\Priya\Desktop\Sem5\DSIP\output_img.jpg',cv2.cvtColor(gray_image,
cv2.COLOR_GRAY2BGR))
```

Output:



Code:

```
cv2.imwrite(r'C:\Users\Priya\Desktop\Sem5\DSIP\rgb_img.jpg',cv2.cvtColor(image_1,cv2.C
OLOR_RGB2BGR))
```

Output:

